

DVWA WEB APPLICATION SECURITY ASSESSMENT & INCIDENT RESPONSE

Name: Harsh Vaidya

Internship: ApexPlanet Software Pvt. Ltd.

Project: DVWA Web Application Security Assessment & Incident Response

Table of Contents

1. Executive Summary	3
2. Introduction	3
3. Objectives	3
4. Scope	3
5. Testing Environment	4
6. Tools Used	4
7. Methodology	4
8. Security Assessment	4
8.1 SQL Injection	4
8.2 Stored Cross-Site Scripting (XSS)	6
8.3 Reflected Cross-Site Scripting (XSS)	7
8.4 Cross-Site Request Forgery (CSRF)	8
9. Findings and Risk Assessment	9
10. Mitigation Recommendations	9
11. Incident Response Simulation	10
11.1 Detection	10
11.2 Containment	10
11.3 Eradication	10
11.4 Recovery	11
11.5 Post-Incident Summary	11
12. Conclusion	11
13. References	11

1. Executive Summary

This project presents a controlled security assessment of the Damn Vulnerable Web Application (DVWA) running in a local Kali Linux laboratory environment. The assessment focused on identifying and demonstrating common web application vulnerabilities through controlled testing and documenting suitable mitigation strategies.

The assessment demonstrated SQL Injection, Stored Cross-Site Scripting (XSS), Reflected XSS, and Cross-Site Request Forgery (CSRF). SQL Injection testing demonstrated manipulation of database queries and exposure of usernames and password hashes. XSS testing demonstrated execution of injected JavaScript, while CSRF testing demonstrated unauthorized execution of a password-change operation in an authenticated session.

An incident-response simulation was also performed using the SQL Injection activity as the simulated security incident. Apache access logs were used to detect the malicious request, followed by documented containment, eradication, and recovery actions.

2. Introduction

Web applications frequently process user-controlled input and perform operations involving databases, sessions, and browser-side content. Improper handling of such input can lead to vulnerabilities that affect confidentiality, integrity, and availability.

DVWA is an intentionally vulnerable web application designed for controlled security testing and learning. It provides a suitable environment for demonstrating common web application vulnerabilities without targeting real-world systems.

This project uses DVWA in a local controlled laboratory environment to demonstrate selected web security weaknesses, assess their impact, and document appropriate defensive measures.

3. Objectives

- Identify selected vulnerabilities in the DVWA web application.
- Demonstrate the impact of SQL Injection, XSS, and CSRF in a controlled environment.
- Capture evidence of successful security testing.
- Document appropriate mitigation and prevention techniques.
- Simulate an incident-response process using a detected SQL Injection request.

4. Scope

The scope of the project was limited to the DVWA web application running locally on Kali Linux. Testing was performed only against the intentionally vulnerable local laboratory application.

Testing areas included SQL Injection, Stored XSS, Reflected XSS, and CSRF. The objective was to identify vulnerabilities, demonstrate their impact in the controlled environment, and document mitigation strategies.

5. Testing Environment

Component	Details
Operating System	Kali Linux
Target Application	Damn Vulnerable Web Application (DVWA)
Target Location	Localhost / 127.0.0.1
Web Server	Apache
Browser	Mozilla Firefox
Testing Environment	Controlled local laboratory

6. Tools Used

- Kali Linux
- DVWA
- Mozilla Firefox
- Apache access/error logs
- Browser-based testing techniques

7. Methodology

The assessment followed a controlled web application security testing workflow. The target application was accessed locally, selected vulnerability classes were tested using controlled inputs, successful results were recorded as evidence, and mitigation recommendations were documented.

For the incident-response portion, the SQL Injection activity was treated as a simulated security incident. The malicious request was detected through Apache access logs, followed by documented containment, eradication, and recovery actions.

8. Security Assessment

8.1 SQL Injection

SQL Injection occurs when user-controlled input is incorporated into a database query in an unsafe manner. During testing, a normal input was first submitted to establish baseline behavior. A Boolean-based SQL Injection payload was then used to alter the query logic and return multiple records.

A UNION-based SQL Injection test was also performed to demonstrate exposure of usernames and password hashes from the application's database.

Test payload examples included:

```
1' OR '1'=1
```

```
1' UNION SELECT user,password FROM users --
```

Firefox browser window showing the DVWA (Damn Vulnerable Web Application) interface. The URL bar displays `http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#`. The page title is "Vulnerability: SQL Injection". The left sidebar contains a menu with options: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (selected), SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, Open HTTP Redirect, Cryptography, API, DVWA Security, PHP Info, and About.

The main content area shows the "Vulnerability: SQL Injection" section. It includes a "User ID:" input field and a "Submit" button. Below the input field, the output displays:

```
ID: 1
First name: admin
Surname: admin
```

Under the "More Information" section, there are links to external resources:

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Firefox browser window showing the DVWA (Damn Vulnerable Web Application) interface. The URL bar displays `http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1'+OR+'1'%3D'1&Submit=`. The page title is "Vulnerability: SQL Injection". The left sidebar contains a menu with options: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection (selected), SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected), XSS (Stored), CSP Bypass, JavaScript Attacks, Authorisation Bypass, Open HTTP Redirect, Cryptography, API, DVWA Security, PHP Info, and About.

The main content area shows the "Vulnerability: SQL Injection" section. It includes a "User ID:" input field and a "Submit" button. Below the input field, the output displays:

```
ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith
```

Under the "More Information" section, there are links to external resources:

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

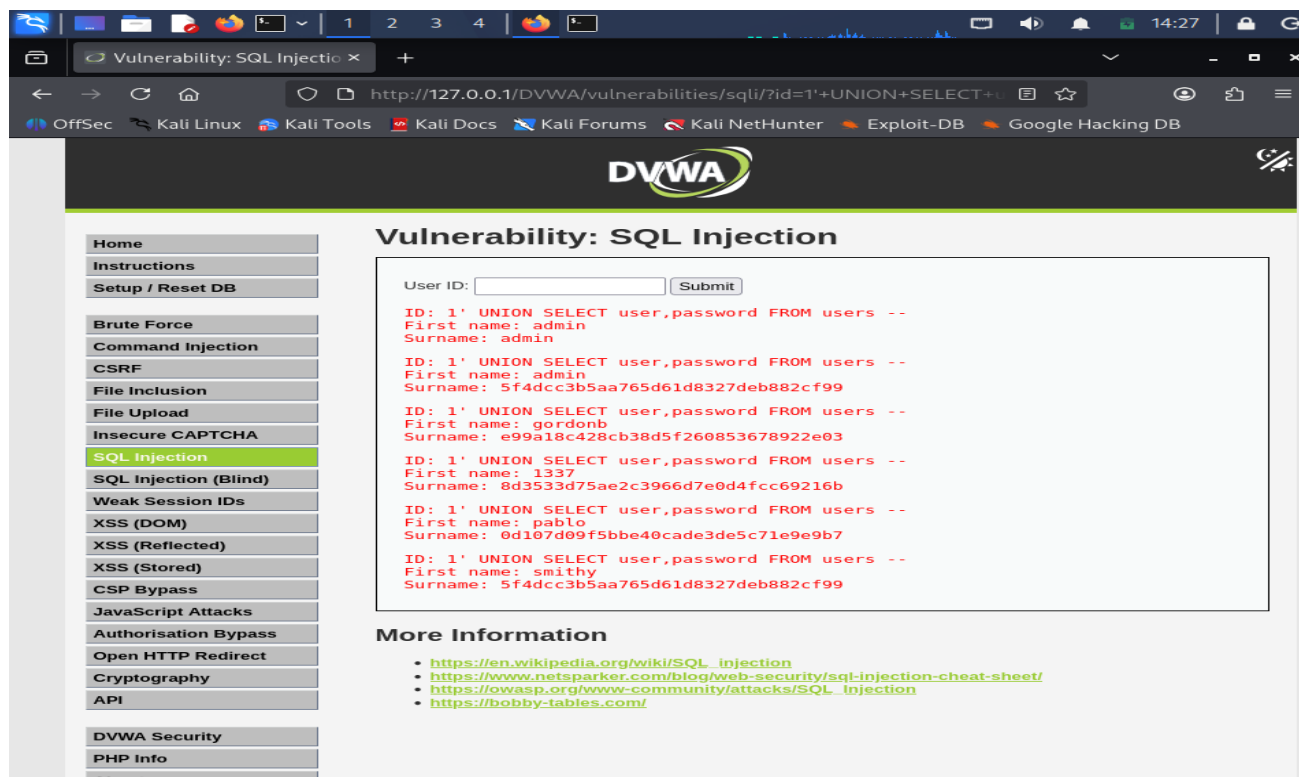


Figure 8.1: SQL Injection testing evidence.

8.2 Stored Cross-Site Scripting (XSS)

Stored XSS occurs when malicious client-side content is stored by an application and subsequently executed when the affected content is viewed. The assessment used a controlled JavaScript payload to demonstrate that injected script content could be stored and executed by the application. Stored XSS commonly occurs in features that accept and store user-generated content, such as comments, messages, profiles, reviews, or forum posts. If the application stores the input without proper validation or sanitization and later displays it without appropriate output encoding, the browser may interpret the stored content as executable JavaScript rather than ordinary text.

In a successful attack, the injected script executes within the security context of the vulnerable web application. Depending on the application's functionality and browser protections, this can potentially allow actions such as manipulating page content, performing actions on behalf of the user, or attempting to access sensitive client-side information.

Impact:

Stored XSS can affect every user who views the compromised content. The vulnerability therefore has a greater potential impact than a reflected attack when malicious content is stored in a page that is frequently accessed.

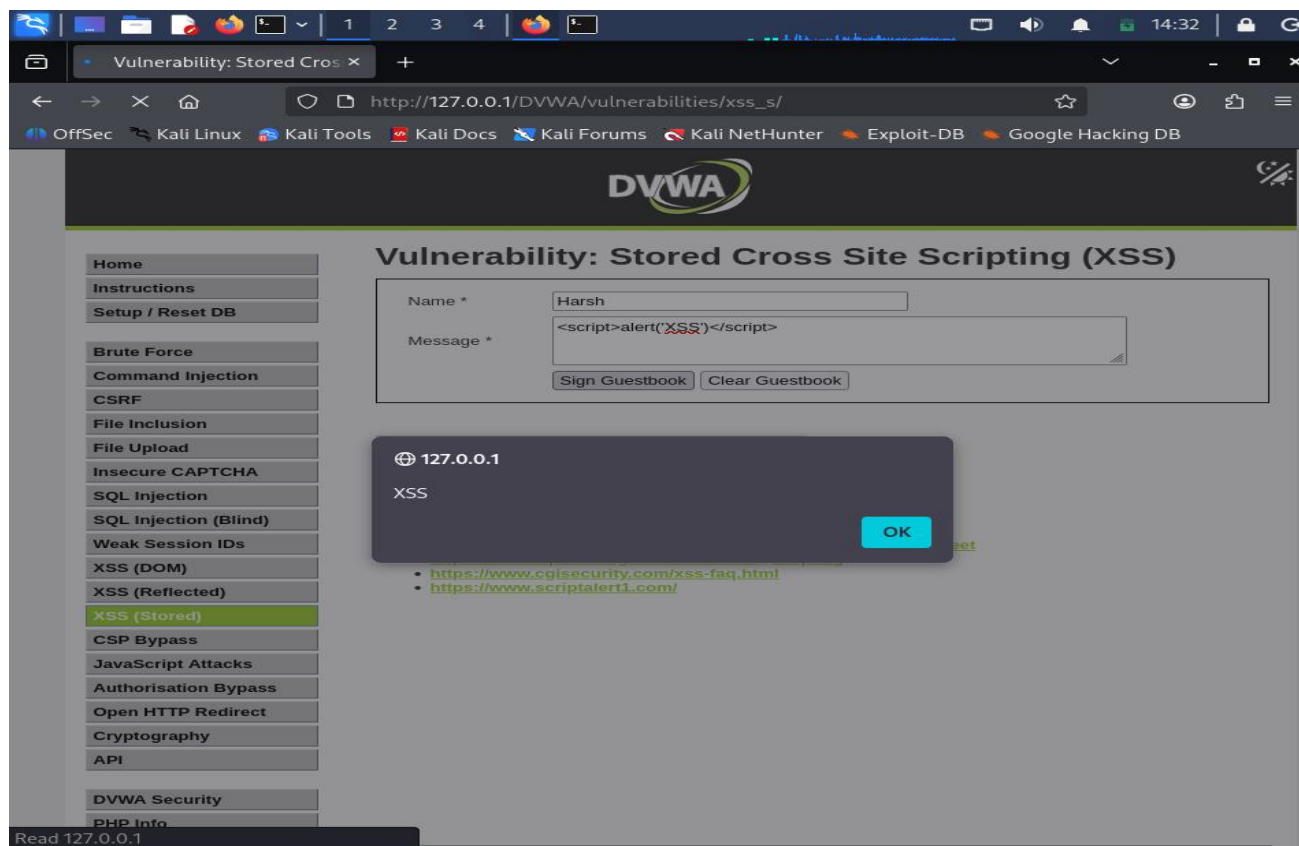


Figure 8.2: Stored XSS testing evidence.

8.3 Reflected Cross-Site Scripting (XSS)

Reflected XSS occurs when user-controlled input is immediately reflected in an application's response without appropriate validation or output encoding. The test demonstrated execution of injected JavaScript through a reflected input.

Reflected XSS commonly occurs when an application takes input from sources such as URL parameters, search fields, or form submissions and directly includes that input in the generated webpage. If the application does not properly handle the input, the browser may interpret the injected content as executable JavaScript.

For an attack to succeed, the victim generally needs to interact with a specially crafted request or URL containing the malicious payload. Because the payload is reflected in the response, the attack is typically associated with a particular request rather than being persistently stored on the server.

Impact:

Reflected XSS can be used to execute attacker-controlled JavaScript in a victim's browser when the victim interacts with a malicious or manipulated request. The vulnerability can therefore affect users who access a crafted link or submit malicious input to the vulnerable functionality.

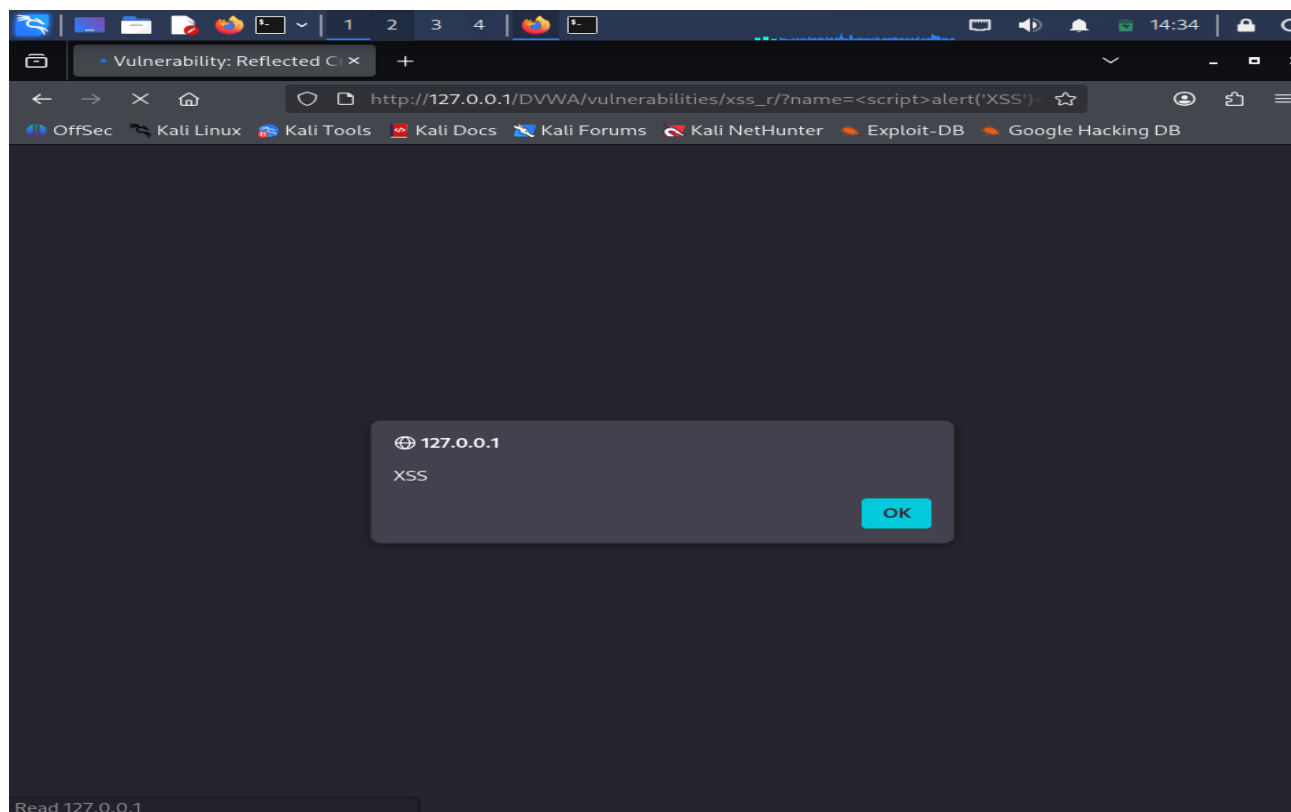


Figure 8.3: Reflected XSS testing evidence.

8.4 Cross-Site Request Forgery (CSRF)

CSRF can cause an authenticated user's browser to perform an unintended state-changing action when appropriate request protections are absent. In the controlled DVWA environment, a crafted request successfully triggered a password-change operation while the user was authenticated.

CSRF mainly affects **state-changing operations**, such as changing a password, updating account information, submitting forms, or performing other actions that modify application data. The vulnerability generally occurs when the application relies only on session cookies for authentication and does not verify whether the request was intentionally generated by the legitimate application interface.

In a typical CSRF attack, an attacker creates a malicious request containing the required parameters for a state-changing operation. When an authenticated victim interacts with the malicious request, the browser may automatically include the victim's session credentials with the request. If the application does not have an additional CSRF protection mechanism, the requested action can be successfully performed.

Impact:

A successful CSRF attack can cause unauthorized changes to a user's account or other application data. The impact depends on which state-changing operations are available to an authenticated user.

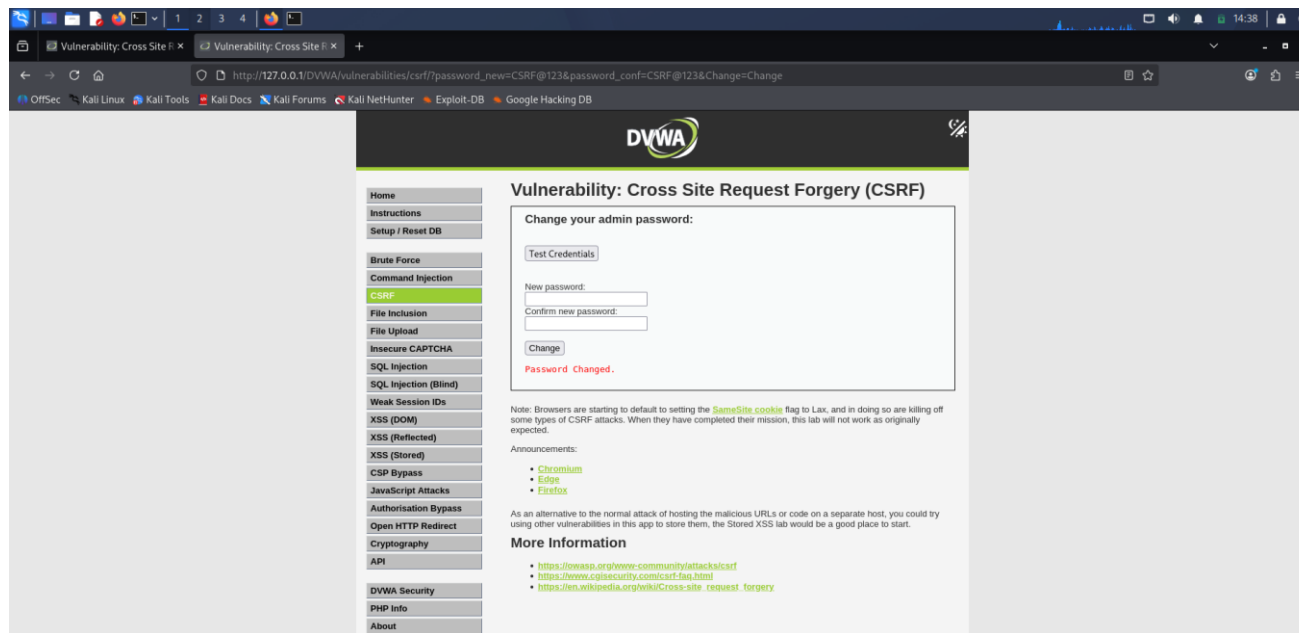


Figure 8.4: CSRF testing evidence.

9. Findings and Risk Assessment

Finding	Severity	Evidence / Impact	Status
SQL Injection	High	Multiple database records and usernames/password hashes exposed.	Identified
Stored XSS	Medium	Injected JavaScript stored and executed by the application.	Identified
Reflected XSS	Medium	Injected JavaScript reflected and executed in the browser.	Identified
CSRF	Medium	Crafted request triggered a password-change operation.	Identified

10. Mitigation Recommendations

SQL Injection: Use prepared statements and parameterized queries. Avoid constructing SQL statements through direct concatenation of user input. Apply appropriate input validation.

Stored XSS: Validate and sanitize input, apply context-appropriate output encoding, and use a suitable Content Security Policy (CSP).

Reflected XSS: Validate input and apply context-appropriate output encoding before reflecting user-controlled data.

11.4 Recovery

Recovery consists of restoring the application to a secure state after remediation, validating normal functionality, repeating the SQL Injection test to verify that malicious input is no longer interpreted as executable SQL, and monitoring application and Apache logs for further malicious activity.

11.5 Post-Incident Summary

The simulated incident was detected through server access logs and followed through documented containment, eradication, and recovery stages. The exercise demonstrates the importance of secure input handling, application logging, monitoring, and structured incident response.

12. Conclusion

The DVWA security assessment successfully demonstrated several common web application vulnerabilities in a controlled local laboratory environment. SQL Injection demonstrated the potential for manipulation of database queries and exposure of sensitive database information. Stored and Reflected XSS demonstrated the risks associated with unsafe handling of client-side content, while CSRF demonstrated the impact of insufficient protection for state-changing requests.

The project also demonstrated a basic incident-response workflow from detection through containment, eradication, and recovery. The documented mitigation strategies provide practical measures for reducing the identified risks in appropriately secured web applications.

13. References

1. **OWASP Foundation — OWASP Top 10: Web Application Security Risks**
2. **OWASP Foundation — SQL Injection Prevention Cheat Sheet**
3. **OWASP Foundation — Cross Site Scripting Prevention Cheat Sheet**
4. **OWASP Foundation — Cross-Site Request Forgery Prevention Cheat Sheet**
5. **OWASP Foundation — Web Security Testing Guide (WSTG)**
6. **PortSwigger Web Security Academy — Web Security Vulnerabilities and Testing Techniques**
7. **Damn Vulnerable Web Application (DVWA) — Official Documentation and Project Repository**
8. **NIST — Computer Security Incident Handling Guide (SP 800-61)**
9. **NIST — Secure Software Development Framework (SSDF), SP 800-218**
10. **Mozilla Developer Network (MDN) — Web Security and HTTP Security Guidelines**